

AI-Assisted Development Workflow

1. Introduction

This document describes how AI was used throughout the development of the Voice-based Retrieval-Augmented Generation (RAG) Assistant. Rather than relying on AI to generate the complete solution, AI was used as an engineering assistant to support research, architecture design, implementation, debugging, optimisation, and testing.

Throughout the project, human judgement remained the primary decision-making mechanism. AI was mainly used to accelerate information gathering, compare technical alternatives, explain unfamiliar concepts, generate implementation examples, and identify potential improvements. Every AI-generated recommendation was reviewed and evaluated before being adopted into the final system.

2. Requirement Analysis

2.1 Why AI used?

At this stage, AI can help analyse business requirements and highlight details that I may have overlooked.

2.2 Human Analysis

Before starting the implementation, I first analysed the project from a business perspective rather than focusing immediately on the technical implementation.

I identified that this project is a **Voice Retrieval-Augmented Generation (Voice RAG) Agent**, where the overall user experience is as important as the accuracy of the generated answers. Therefore, instead of only considering functional requirements, I first identified the expectations of a real-world enterprise application.

The initial business requirements included:

- A natural and smooth voice conversation experience.
- Fast response time to minimise waiting during voice interactions.
- Accurate document retrieval and answer generation.

- The ability to utilise conversation history to provide more context-aware responses.
- An intuitive and user-friendly interface that resembles a real production application.
- A simple and convenient user interaction workflow.

These requirements were documented before consulting AI and later served as the foundation for subsequent technology selection, architecture design, and system optimisation.

2.3 AI Assistance

After completing my own business requirement analysis, I used AI to review the identified requirements and determine whether additional enterprise considerations had been overlooked.

Instead of asking AI to define the project requirements, I provided the project background together with my own analysis and requested AI to review the proposed business objectives from the perspective of an experienced software architect.

AI identified several additional requirements that should be considered for a production-ready Voice RAG system, including:

- **Retrieval Reliability** – Responses should include citations so that users can verify the source of the generated answers and improve overall trustworthiness.
- **Robustness** – The system should continue operating reliably under noisy environments to ensure stable speech recognition performance.
- **Long Document Support** – The application should support users uploading large documents while maintaining stable retrieval performance in real-world scenarios.

These additional requirements complemented my initial analysis and expanded the project scope beyond basic functionality.

2.4 Prompt Strategy

Rather than asking a general question such as:

"How should I build this project?"

I provided AI with:

- The project background.
- My own business requirement analysis.
- The expected user experience.
- The intended deployment scenario.
- The overall project objectives.

I then asked AI to review these requirements, identify missing considerations, and explain why each additional requirement would be important in a real enterprise environment.

Providing detailed project context enabled AI to perform requirement validation instead of generating generic development suggestions.

2.5 Prompt Evolution

Version 1

The initial prompt asked AI to review my business requirement analysis and identify any missing functional or non-functional requirements.

Although AI successfully identified several additional considerations, the responses remained relatively high-level and lacked sufficient justification.

Version 2

The prompt was refined by assigning AI the role of an experienced enterprise software architect.

Additional constraints were introduced, requiring AI to evaluate the project from the perspective of:

- Enterprise software development.
- Real-world business scenarios.
- Voice-based user experience.
- Long-term maintainability.
- Production deployment.

Rather than simply listing additional requirements, AI explained why each requirement would improve the overall user experience and how it would influence subsequent design decisions.

2.6 Human Review

After reviewing AI's recommendations, I evaluated whether each additional requirement aligned with the project's objectives.

Requirements that directly contributed to improving the user experience, system reliability, and real-world usability were incorporated into the development plan and later influenced the technology selection, architecture design, optimisation, and testing strategies.

2.7 Outcome

Combining my own business analysis with AI-assisted requirement validation resulted in a more comprehensive understanding of the project before implementation began.

Rather than defining the project direction, AI acted as a logical reviewer, helping to identify overlooked requirements and expand the analysis from a business perspective.

The final requirement analysis established the development principles used throughout the project, ensuring that future design decisions were driven not only by technical considerations but also by the intended user experience and real-world deployment requirements.

3. Technology Research

3.1 Why AI used?

The AI has been trained on a vast amount of code and is capable of searching for state-of-the-art technologies; however, as I am not very familiar with the use of RAG, I would like to ask the AI for advice on the most appropriate way to utilise this technology.

3.2 Human Analysis

Before selecting any technologies, I first analysed the functional requirements and non-functional requirements of the project. Rather than focusing solely on model

performance, I identified the business requirements that each technology needed to satisfy.

These considerations included:

- Low response latency.
- Local deployment capability.
- Ease of maintenance.
- Scalability.
- Cost.
- Ease of integration.
- Suitability for a browser-based Voice RAG application.

These requirements became the evaluation criteria before consulting AI.

3.3 AI Assistance

Instead of asking AI to recommend a single technology, AI was used to perform structured comparisons between multiple candidate technologies.

For each system component, AI was instructed to recommend several suitable technologies, explain their advantages and disadvantages, and analyse the trade-offs from an engineering perspective.

Examples included:

Vector Database	ChromaDB, FAISS, Pinecone
Speech Recognition	Faster Whisper, Google Speech-to-Text, Deepgram
Embedding Model	Gemini Embedding, BAAI BGE
Reranker	BGE Reranker, Cohere Rerank
Large Language Model	Gemini, OpenAI, Ollama
Text-to-Speech	Kokoro, ElevenLabs
Vector Database	ChromaDB, FAISS, Pinecone

Rather than identifying the "best" technology, AI explained the strengths, weaknesses, deployment requirements, and suitable use cases for each option.

3.4 Prompt Strategy

The prompts focused on engineering comparisons instead of asking AI for direct recommendations.

By telling the AI what I require from each technology, the AI can provide a more comprehensive answer

Typical prompts requested AI to compare technologies according to:

- Advantages
- Disadvantages
- Deployment complexity
- Latency
- Cost
- Scalability
- Ease of maintenance
- Suitable use cases

This produced objective comparisons instead of subjective recommendations.

3.5 Human Review

After reviewing the comparison results, the final technology was selected according to the project's business requirements rather than simply choosing the highest-performing solution.

For example, when selecting the vector database, AI compared cloud-based solutions such as Pinecone with local vector databases.

Although cloud-based services provide managed infrastructure and easier scalability, the project prioritised response latency and avoided unnecessary network communication.

Therefore, a local vector database was selected because it provides faster retrieval, lower latency, and simpler deployment for the scope of this project.

The same decision-making process was applied throughout the technology selection phase, balancing engineering trade-offs instead of selecting technologies solely based on benchmark performance.

3.6 Outcome

AI significantly accelerated the technology research process by producing structured comparisons between multiple alternatives.

However, the final technology stack was determined through manual evaluation based on the project's business requirements, engineering constraints, and deployment objectives rather than AI recommendations alone.

4. Architecture Design

4.1 Why AI used?

The design of truly enterprise-level project structures is highly complex and difficult to understand; AI can assist with searching, thereby significantly improving efficiency and professionalism.

4.2 Human Analysis

Before generating the system architecture, I first defined the software engineering objectives of the project.

Instead of requesting AI to create an arbitrary architecture, I determined the architectural requirements based on the project background and future maintenance needs.

The architecture needed to:

- Support modular development.
- Be easy to maintain.
- Separate frontend and backend responsibilities.
- Clearly separate AI services.
- Reduce coupling between components.
- Support future feature expansion.
- Follow enterprise software engineering practices.

Only after these requirements were clearly defined did I begin using AI.

4.3 AI Assistance

Rather than directly asking AI to generate a system architecture, AI was first asked to create a professional architecture design prompt.

The purpose was to transform my development requirements into a prompt that clearly described:

- Project background.
- Functional requirements.
- Technology stack.
- Software engineering principles.
- Enterprise development practices.
- Maintainability requirements.
- Scalability requirements.
- Expected architectural outputs.

The generated professional prompt was then submitted to an AI agent to generate the complete system architecture.

4.4 Prompt Strategy

Instead of writing architecture prompts manually, AI assisted in improving the prompt itself.

The final prompt instructed the architecture agent to consider:

- Enterprise architecture.
- Layered design.
- Separation of concerns.
- Low coupling.
- High cohesion.
- Maintainability.
- Scalability.
- Future extensibility.
- Clear module responsibilities.
- API interaction.

- Deployment considerations.

Providing detailed engineering constraints produced architecture proposals that were significantly more practical than generic software architecture diagrams.

4.5 Human Review

After the architecture had been generated, AI was further asked to explain the rationale behind each architectural decision.

This included explanations of:

- Why individual layers were separated.
- Why each service existed.
- Why modules interacted in specific ways.
- How maintainability was improved.
- How future scalability was supported.

Understanding these design decisions allowed me to evaluate whether the architecture aligned with the project requirements before accepting it.

Where necessary, unnecessary complexity was removed and the architecture was simplified before implementation.

4.6 Outcome

Using AI in two stages—first to generate a high-quality architecture prompt, and then to generate the architecture itself—produced a significantly more structured software design.

Rather than accepting the generated architecture directly, every design decision was reviewed against the project's engineering objectives before being adopted, ensuring that the final architecture remained practical, maintainable, and aligned with enterprise software engineering principles.

5. Development

5.1 Why AI used?

AI can help write and optimise code, as well as fix bugs. This can help save time and improve efficiency.

5.2 Human Analysis

After completing the system architecture, I planned the implementation sequence before writing any code. Instead of developing all modules simultaneously, I divided the project into several independent stages so that each component could be implemented, tested, and integrated progressively.

This approach allowed me to validate each module individually before integrating it into the complete Voice RAG pipeline.

5.3 AI Assistance

PDF Processing

Since PDF parsing was the first unfamiliar component, I first asked AI where development should begin to ensure that the implementation aligned with the designed architecture.

AI recommended starting with **PDF parsing**, as it forms the foundation of the Retrieval-Augmented Generation (RAG) pipeline. As I had limited experience with PDF processing, I then asked AI to provide a reference implementation to help me understand the overall parsing workflow and system structure.

After understanding the workflow, I further asked AI to generate example backend APIs, frontend interactions, and a simple end-to-end pipeline as learning references. These examples helped me understand how different modules communicate with each other before implementing my own solution.

During implementation, whenever errors occurred, I provided AI with the relevant file locations, source code, and error messages. Instead of directly asking for a fix, I asked AI to explain the root cause of the issue so that I could understand the framework behaviour before modifying the code myself.

Embedding and Vector Storage

After completing PDF parsing, I moved on to the embedding pipeline.

Since I was unfamiliar with the implementation sequence, I asked AI which component should be developed first.

AI recommended implementing the **embedding model first**, followed by integrating it into the **document service**. I also discussed vector storage strategies with AI before deciding to store vectors locally during development to simplify deployment and reduce retrieval latency.

Retrieval Pipeline

After establishing the embedding pipeline, I focused on improving document retrieval.

I asked AI about:

- How many documents should be returned by the retriever before reranking.
- Which reranker model provides the best retrieval performance.
- How reranking should be integrated into the retrieval pipeline.

These discussions helped me better understand the retrieval workflow before implementing the final RAG pipeline.

Large Language Model (LLM)

For the LLM component, I asked AI to compare available models and recommend those that provide the best balance between capability, free usage limits, and practical value for this assignment.

The comparison helped me evaluate different models before selecting the final LLM for the project.

Development Workflow Planning

After completing the initial RAG implementation, I was uncertain whether I should continue improving the retrieval pipeline or complete the entire voice interaction workflow first.

I consulted AI regarding the most efficient development strategy.

AI recommended completing the complete Voice RAG workflow first before returning to optimise the RAG pipeline. This approach establishes a working foundation early, allowing the complete application to be validated before introducing additional retrieval improvements. I adopted this recommendation as it provided a clearer development roadmap.

Frontend Development

Rather than spending significant time designing the user interface at the beginning of the project, I first asked AI to generate a professional frontend interface that could serve as the foundation of the application.

The objective of the initial interface was to provide a functional and visually appealing starting point while allowing further refinement after the core Voice RAG workflow had been completed.

The frontend was then progressively improved through multiple prompt iterations during later stages of development.

Audio Processing

To improve speech recognition quality, I asked AI how the audio processing pipeline could be enhanced before transcription.

AI recommended several preprocessing techniques, including:

- Noise cancellation.
- Voice enhancement.
- Sample rate optimisation.

These suggestions were reviewed before deciding which techniques were suitable for the project.

5.4 Human Review

Throughout the implementation phase, AI was primarily used as a development assistant rather than a code generator.

Reference implementations, architectural suggestions, and debugging explanations provided by AI were manually reviewed before being incorporated into the project.

Whenever multiple implementation approaches were suggested, I evaluated each option based on maintainability, implementation complexity, compatibility with the existing architecture, and future extensibility before deciding which approach to adopt.

5.5 Outcome

Using AI during implementation significantly accelerated the learning process for unfamiliar technologies while reducing development time.

Instead of relying on AI to generate the complete application, I used it to explain concepts, recommend implementation sequences, provide reference examples, and assist with debugging. Final implementation decisions remained under my control, ensuring that every module aligned with the overall system architecture and project requirements.

6. AI-assisted Iterative Development

6.1 Why AI used?

AI can assist with decision-making regarding which steps to carry out first, thereby helping to avoid efficiency losses caused by errors in the development process.

6.2 Development Strategy

Rather than following a fixed development roadmap, the project adopted an iterative development strategy. After completing each development stage, I first evaluated the current implementation and then consulted AI to determine the most appropriate next step.

Instead of allowing AI to make the decisions, I first analysed the current progress and identified the available development options. AI was then asked to review the current state of the project, recommend the next development priority, and explain the reasoning behind its recommendation. After reviewing the proposed approach, I decided whether it aligned with the project objectives before proceeding.

This iterative workflow ensured that the project evolved progressively, with each stage building upon the previous one while maintaining a clear development direction.

6.3 Stage 1 – PDF Parser Development

The first development stage focused on implementing the PDF parser.

Before writing any code, I asked AI which module should be developed first based on the overall system architecture. AI recommended starting with the PDF parser because it has no dependencies on other system components and serves as the foundation of the Retrieval-Augmented Generation (RAG) pipeline.

Following this recommendation, I implemented the PDF parser first, allowing the document processing workflow to be established before developing chunking, embedding generation, and document indexing.

6.4 Stage 2 – Core RAG Development

After completing the PDF parser, I again consulted AI regarding the next development priority.

AI recommended implementing the core RAG pipeline by following the natural document retrieval workflow. This included document chunking, embedding generation, vector storage, document retrieval, reranking, and Large Language Model (LLM) integration.

At this stage, the primary objective was to establish a complete and functional retrieval pipeline rather than focusing on optimisation.

6.5 Stage 3 – Completing the End-to-End Workflow

Once the core RAG pipeline was functional, I considered whether to begin improving retrieval accuracy or complete the remaining application components.

I asked AI which development direction would be more appropriate.

AI recommended developing the frontend first so that a complete end-to-end Voice RAG application could be established before introducing further optimisation. This allowed the entire workflow to be validated through a functional user interface instead of optimising an incomplete system.

After completing the frontend, I again consulted AI regarding the next development stage.

Rather than immediately optimising the RAG pipeline, AI suggested implementing the Text-to-Speech (TTS) module first to complete the voice interaction workflow. This

ensured that the complete user journey—from speech input to spoken response—could be validated before additional optimisation work began.

6.6 Stage 4 – RAG Optimisation

After the complete Voice RAG workflow had been established, the development focus shifted towards improving retrieval quality and overall system performance.

Before requesting optimisation suggestions, I first analysed the limitations of the existing implementation and identified areas that could be further improved.

I then used a professionally designed prompt, asking AI to assume the role of an experienced RAG engineer and enterprise software architect. The prompt instructed AI to evaluate the current system while considering:

- Enterprise software engineering practices.
- Business requirements.
- Low-latency voice interaction.
- High retrieval accuracy.
- Maintainability.
- Scalability.
- Real-time response performance.

Based on these requirements, AI analysed the current implementation, identified potential weaknesses, and proposed optimisation strategies together with the reasoning behind each recommendation.

After reviewing these suggestions, I selected the optimisation techniques that best matched the project requirements. The detailed optimisation process is discussed in the following **Optimisation** chapter.

6.7 Stage 5 – Testing and Continuous Improvement

After completing the optimisation stage, the project entered the testing phase.

The completed system was evaluated using retrieval benchmarks, voice performance evaluation, latency measurements, load testing, and manual validation.

Rather than treating testing as the final step of development, the testing results were analysed to identify remaining weaknesses within the system. These findings were then

used to guide further improvements, allowing the system to be refined continuously after each evaluation.

This iterative process ensured that development, optimisation, and testing formed a continuous improvement cycle throughout the project, rather than being treated as isolated development stages.

7. Optimisation

7.1 Why AI used?

AI can help analyse business requirements and provide recommendations based on user needs

7.2 Human Analysis

After completing the initial implementation of the Voice RAG system, I shifted my focus from adding new features to improving the overall system performance and user experience.

Although the application was fully functional, I wanted to identify potential improvements in retrieval quality, response latency, deployment flexibility, and overall system architecture.

Instead of relying solely on my own understanding, I used AI to review the existing implementation and identify possible optimisation opportunities.

7.3 AI Assistance

Improving the RAG Pipeline

I first asked AI how the existing RAG architecture could be further improved to increase retrieval accuracy.

AI suggested several possible improvements, including:

- Query expansion to improve retrieval for short or ambiguous queries.
- Metadata filtering to narrow the retrieval scope.

- Hybrid search to combine multiple retrieval strategies.
- Fallback mechanisms when no relevant documents are retrieved.
- Agent RAG for handling more complex queries.
- Self-RAG for validating generated answers.

System Optimisation

After discussing retrieval improvements, I asked AI to review the overall system and identify additional optimisation opportunities.

AI analysed the current implementation and suggested several possible improvements, including:

- Removing Self-RAG to reduce unnecessary LLM calls.
- Replacing the Agent RAG classifier with a lightweight routing mechanism.
- Replacing or skipping the reranker to reduce response time.
- Caching frequently used queries.
- Using cloud-based embedding models.
- Using cloud-based Whisper services for faster speech recognition.

These suggestions focused primarily on reducing latency while maintaining acceptable retrieval quality.

Large PDF Processing

During development, I encountered performance issues when processing large PDF documents.

Uploading large documents generated a significant number of chunks, resulting in increased memory usage and slower processing.

I consulted AI regarding possible solutions, and AI recommended processing PDF documents in smaller batches instead of processing the entire document at once.

This batching strategy reduced memory usage while maintaining stable document processing.

Prompt Optimisation

I also asked AI to review the prompt used by the Voice RAG system and suggest improvements.

AI identified several issues affecting response quality, including:

- Slow response generation.
- Unnecessary emojis and special characters in generated responses.
- Lack of conversation history.
- Prompt structure that could better match user questions.
- Missing document citations.

AI then suggested modifying the prompt to improve conversation quality and include citations within the generated responses.

7.4 Human Review

Rather than implementing every optimisation suggested by AI, I evaluated each recommendation based on the project's engineering objectives.

The review considered:

- Expected improvement in retrieval quality.
- Response latency.
- Additional implementation complexity.
- API cost.
- Suitability for the assignment.
- overall user experience.

Only the optimisations that provided practical benefits without introducing unnecessary complexity were incorporated into the project.

7.5 Outcome

Using AI during the optimisation stage enabled me to evaluate the existing implementation from multiple engineering perspectives.

Instead of simply identifying problems, AI provided practical optimisation suggestions together with their expected trade-offs, allowing me to make informed engineering decisions before modifying the system.

The final implementation therefore reflects a balance between retrieval accuracy, response latency, maintainability, and development complexity, rather than maximising a single performance metric.

8. AI-assisted Testing

8.1 Why AI used?

Testing is a tedious task, and the use of AI can significantly improve efficiency.

8.2 Human Analysis

After completing the optimisation stage, the project entered the testing phase.

Before selecting any testing methods, I first analysed what aspects of the system should be evaluated to determine whether the Voice RAG agent met the original business requirements.

Based on the business analysis conducted at the beginning of the project, I identified that testing should focus on evaluating the user experience rather than simply verifying whether the system functioned correctly.

The key testing objectives included:

- Speech recognition accuracy.
- Response latency during voice interactions.
- Speech recognition performance under noisy environments and overlapping speakers.
- Retrieval accuracy of the RAG pipeline.
- Overall answer quality.
- End-to-end user experience.

These objectives became the foundation for designing the testing strategy.

8.3 AI Assistance

After defining the testing objectives, I consulted AI to determine which evaluation methodologies would be most suitable for measuring the performance of a Voice RAG system.

Rather than asking AI how to test the application in general, I used a professional prompt instructing AI to assume the role of an experienced RAG evaluation engineer.

The prompt requested AI to recommend comprehensive evaluation methods capable of measuring retrieval quality while ensuring that the testing process covered the entire knowledge base and represented realistic enterprise use cases.

AI proposed several evaluation frameworks and methodologies, including:

- RAGAS
- DeepEval
- Custom retrieval evaluation
- Manual evaluation

For each framework, AI explained its advantages, disadvantages, implementation complexity, and suitable application scenarios.

8.4 Human Review

After reviewing the proposed evaluation methods, I compared each framework according to the project requirements.

Since this project primarily focused on evaluating Retrieval-Augmented Generation performance, I selected **RAGAS** as the primary evaluation framework because it was specifically designed for RAG systems and provides dedicated retrieval evaluation metrics.

This selection was considered more appropriate than using general-purpose evaluation frameworks for the objectives of this project.

8.5 Benchmark Dataset Generation

After selecting the evaluation framework, the next challenge was constructing an evaluation dataset.

Instead of manually creating benchmark questions, I provided the complete project document to AI and instructed it to generate representative testing questions based on the document content.

The prompt required AI to:

- Cover all sections of the document.
- Generate realistic enterprise-style user questions.
- Include different levels of question difficulty.
- Ensure that every question could be answered using the uploaded document.

This significantly reduced the time required to prepare the benchmark dataset while providing broad coverage of the knowledge base.

After generating the benchmark questions, AI was further instructed to generate the corresponding ground-truth answers based solely on the document content. These question-answer pairs were then reviewed before being used during evaluation.

8.6 Result Analysis

After completing the evaluation, I analysed the testing results to identify potential weaknesses within the system.

Rather than interpreting the results independently, I provided the evaluation outputs to AI and requested an analysis of the overall performance.

AI reviewed the testing metrics, identified areas that could be improved, and suggested possible optimisation strategies for retrieval quality and overall system performance.

These recommendations were then evaluated before deciding whether additional modifications should be made to the system.

8.7 Outcome

Throughout the testing phase, AI acted as an evaluation consultant rather than an automated testing tool.

AI assisted in selecting appropriate evaluation methodologies, generating benchmark datasets, constructing ground-truth answers, and analysing testing results. Final decisions regarding the testing strategy and subsequent system improvements remained under my control.

The testing process therefore became an iterative cycle, where evaluation results were continuously reviewed and used to guide further refinements, ensuring that the final Voice RAG agent met the project's business requirements for both retrieval quality and user experience.

9. Continuous Improvement Based on Testing Results

Testing was not treated as the final stage of the project. Instead, the evaluation results were analysed to identify performance bottlenecks, and AI was consulted to recommend possible engineering improvements before deciding whether they should be implemented.

One of the major issues identified during testing was the relatively high response latency of the Text-to-Speech (TTS) pipeline. Since the project is a voice-based question answering system, response speed directly affects the overall user experience.

After identifying this issue, I asked AI to analyse the current implementation and recommend possible optimisation strategies. Rather than suggesting a single solution, AI proposed several alternatives together with their engineering trade-offs.

Switching to a Faster TTS Model

AI first suggested replacing the existing TTS model with a faster alternative.

Although this approach could reduce speech generation time, it would also noticeably reduce the naturalness and quality of the generated voice. Since providing a natural conversational experience was one of the project's original business requirements, I decided not to adopt this solution.

Caching Generated Audio

AI also suggested caching previously generated TTS audio so that identical responses could be returned immediately without regenerating speech.

After reviewing this approach, I decided not to implement it because users in a real-world Voice RAG application rarely ask exactly the same question repeatedly. Therefore, the cache hit rate would likely be low while introducing additional implementation complexity.

Audio Streaming and Natural Speech Delivery

AI suggested that, instead of waiting for the entire response to be synthesised before playback, the system should begin speech generation as soon as sufficient text becomes available. The objective was to reduce the perceived waiting time and create a more natural conversational experience.

After reviewing the recommendation, I refined the existing streaming implementation by introducing sentence-boundary streaming. Rather than sending the complete LLM response to the TTS engine, completed sentences were detected and immediately forwarded for speech synthesis while the remaining text continued to stream from the LLM.

This approach allowed audio playback to begin before the entire response had been generated, significantly reducing the perceived response latency and producing a more natural conversation flow. To improve stability, a debounce mechanism and force-flush timeout were also introduced to handle long responses that contained few punctuation marks.

Voice Configuration

AI also suggested reviewing the speech synthesis configuration to improve the overall listening experience.

Based on this recommendation, the TTS configuration was refined by selecting a more natural multilingual neural voice while maintaining a neutral speaking rate and pitch. This produced more natural speech without sacrificing intelligibility or increasing response latency.

Text Pre-processing

To further improve speech quality, AI recommended preprocessing the generated responses before passing them to the TTS engine.

A text-cleaning stage was therefore introduced to remove unnecessary Markdown formatting while preserving meaningful symbols such as percentages and currency values. This ensured that numerical values and units were pronounced naturally and reduced unnecessary interruptions during speech synthesis.

Parallel Processing

AI also suggested increasing the level of parallelism within the pipeline to reduce the overall response time.

Rather than processing every stage sequentially, opportunities for overlapping text generation and speech synthesis were explored where possible. Since this optimisation improved performance without negatively affecting answer quality, it was identified as a promising direction for further optimisation.

Retrieval Pipeline Optimisation

After implementing the above improvements, the response latency remained higher than expected.

I therefore consulted AI again to identify additional performance bottlenecks.

AI suggested reducing the reranker workload by lowering the retrieval **top-k** value and decreasing the number of document chunks passed to the reranker.

After evaluating the trade-off between retrieval accuracy and response speed, these parameters were adjusted to improve overall responsiveness while maintaining acceptable retrieval quality.

API Rate Limit Analysis

Despite the retrieval optimisations, testing still showed inconsistent response latency.

Further investigation revealed that the primary bottleneck was not the application itself but the external API rate limits imposed by the LLM service.

To obtain more reliable benchmarking results, the automated testing process was modified to introduce a **30-second interval** between consecutive evaluation requests, preventing temporary API throttling during testing.

In addition, backend rate limiting was implemented to prevent excessive requests in a real deployment environment, reducing the likelihood of triggering API limits while improving overall system stability.

Outcome

Rather than applying every optimisation suggested by AI, each recommendation was evaluated according to the project's business requirements, particularly the need to deliver a responsive and natural voice interaction experience.

AI acted as an engineering advisor by proposing multiple optimisation strategies and explaining their trade-offs, while the final implementation decisions were made after considering response latency, answer quality, implementation complexity, and real-world usage scenarios.